

Retomando Definiciones



¿Qué era la POO?



¿Qué era la POO?

Es un modo o **paradigma de programación**, que nos permite **organizar el código pensando el problema como una relación entre “cosas”,** denominadas **objetos**. Los objetos **se trabajan utilizando las clases**. Estas nos permiten **agrupar un conjunto de variables y funciones** que veremos a continuación.



Atributos

Cosas de lo más cotidianas como una computadora o un coche pueden ser representadas con

clases. Estas clases tienen diferentes características, que

en el caso del computadora podrían ser la marca o modelo.

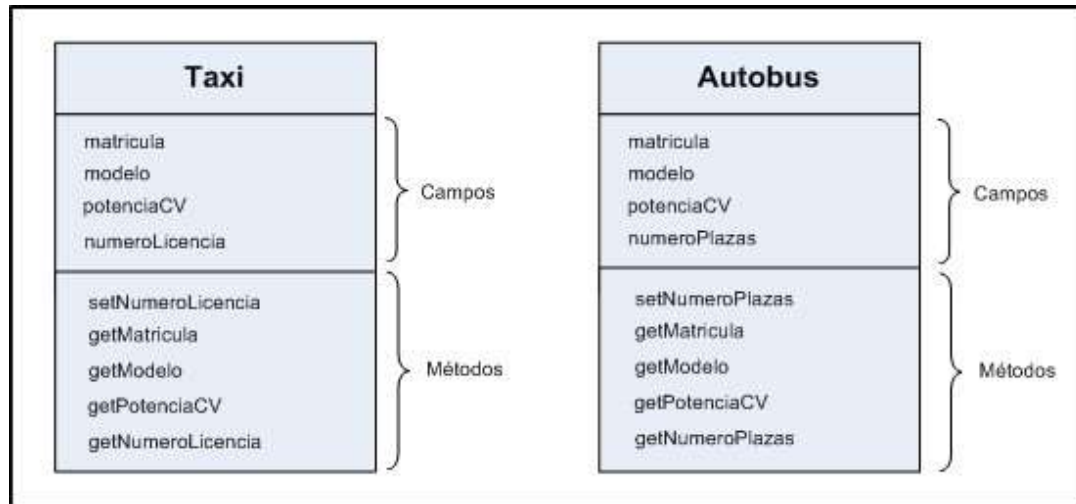
Llamaremos a estas

características, **atributos.**



Métodos

Por otro lado, las **clases** tienen un conjunto de funcionalidades. En el caso de la computadora podría ser imprimir o reproducir. Llamaremos a estas funcionalidades **métodos**.



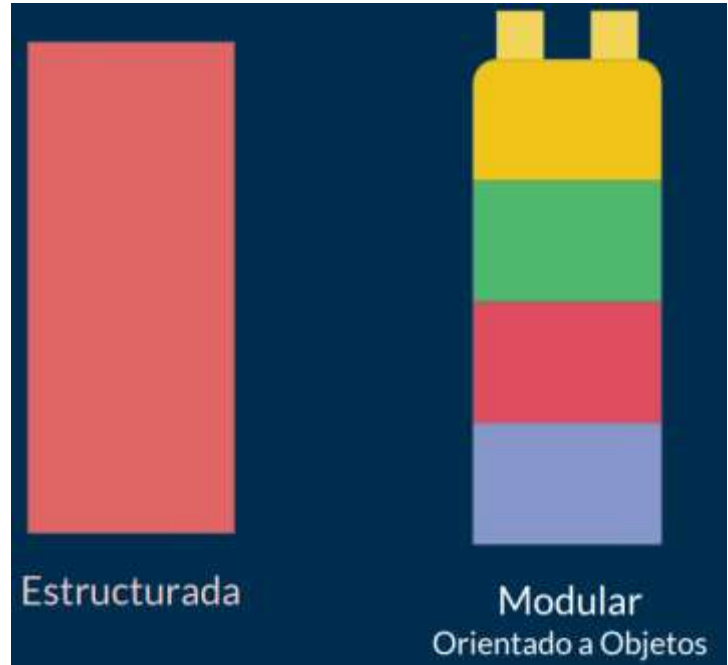
Objetos

Por último, pueden existir diferentes tipos de computadoras. Llamaremos a estos diferentes tipos de computadoras objetos. Es decir, **el concepto abstracto de computadora es la clase, pero cualquier tipo de computadora particular será el objeto.**

¿Para qué usar la POO?

En el mundo de la programación hay gran cantidad de aplicaciones que realizan tareas muy similares y es importante identificar los patrones que nos permiten no reinventar la rueda. La programación orientada a objetos intentaba resolver esto.

Fuente: elibrodepython.com





Python está diseñado para soportar el paradigma de la **programación orientado a objetos (POO)**, donde **todos los módulos se tratan como entidades que tienen atributos, y comportamiento**. Lo cual, en muchos casos, ayuda a crear programas de una manera sencilla, además de que los componentes son reutilizables.

Clases



Uso de POO: Clases

Lo primero es crear una clase, a la cual llamaremos “perro”. Se trata de una clase vacía y sin mucha utilidad práctica, pero es la mínima clase que podemos crear. El uso del PASS realmente no tiene utilidad, pero daría un error si después de los : no tenemos contenido.

```
1 class Perro:  
2     pass
```



Instanciar una clase

Ahora que tenemos la clase, podemos crear uno o más objeto de la misma, como si de una variable normal se tratase.

Nombre de la variable igual a la clase con (). Dentro de los paréntesis irían los parámetros de entrada si los hubiera.

```
class Perro:  
    pass  
  
perro1 = Perro()  
perro2 = Perro()
```

Atributos

Asignando atributos

A la hora de añadir atributos a la clase, es importante distinguir dos tipos:

1. **Atributos de instancia:** Pertenecen a la instancia de la clase o al objeto. Son atributos particulares de cada instancia, en nuestro caso de cada perro.
2. **Atributos de clase:** Se trata de atributos que pertenecen a la clase, por lo tanto serán comunes para todos los objetos.



Asignando atributos de Instancia

Empecemos creando el nombre y la raza del perro. Esto se suele hacer por medio de un método `__init__` que será llamado automáticamente cuando se cree un objeto. No se le puede cambiar el nombre y tiene su definición propia,

```
class Perro:

    #Constructor de la clase
    def __init__(self, nombre, raza):

        #Atributos de la instancia
        self.nombre = nombre
        self.raza = raza
```

¿Qué es el self?

Es una variable que representa la instancia de la clase, y deberá estar siempre ahí.

El uso de `__init__` y el doble `__` no es una coincidencia. Cuando veas un método con esa forma, significa que está reservado para un uso especial del lenguaje.

En este caso sería lo que se conoce como **constructor**.



Asignando atributos de clase

Hasta ahora hemos definido atributos de **instancia**, ya que **pertenecen a cada perro concreto**.

Ahora vamos a definir un **atributo de clase**, que será **común para todos los perros**.

Por ejemplo, la especie de los perros es algo común para todos los objetos *Perro*.

```
class Perro:

    #Atributos de clase
    especie = "Mamífero"

    #Constructor de la clase
    def __init__(self, nombre, raza):

        #Atributos de la instancia
        self.nombre = nombre
        self.raza = raza
```



Asignando atributos de clase

Dado que es un atributo de clase, no es necesario crear un objeto para acceder al atributo.

```
print(Perro.especie)  
# mamífero
```

Se puede acceder también al atributo de clase desde el objeto.

```
mi_perro = Perro("Toby", "Bulldog")  
mi_perro.especie  
# 'mamífero'
```

- *De esta manera, todos los objetos que se creen de la clase **perro** compartirán ese atributo de clase.*



```
14 class Perro:
15
16     #Atributos de clase
17     especie = "Mamífero"
18
19     #Constructor de la clase
20     def __init__(self, nombre, raza):
21
22         #Atributos de la instancia
23         self.nombre = nombre
24         self.raza = raza
25
26
27 perro1 = Perro("Sammy", "Caniche")
28
29 print(f"Su nombre es: {perro1.nombre}")
30 print(f"Su raza es: {perro1.raza}")
31 print(f"Es un: {perro1.especie}")
```

Estructura final de una clase con atributos

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

Su nombre es: Sammy
Su raza es: Caniche
Es un: Mamífero

Métodos

Definiendo métodos

En realidad cuando usamos `__init__` ya estábamos definiendo un método



A continuación vamos a ver cómo definir métodos que le den alguna funcionalidad interesante a nuestra clase, siguiendo con el ejemplo de perro.





Definiendo métodos

```
class Perro:
    # Atributo de clase
    especie = 'mamífero'

    # El método __init__ es llamado al crear el objeto
    def __init__(self, nombre, raza):
        print(f"Creando perro {nombre}, {raza}")

        # Atributos de instancia
        self.nombre = nombre
        self.raza = raza

    def ladra(self):
        print("Guau")

    def camina(self, pasos):
        print(f"Caminando {pasos} pasos")
```

Vamos a codificar dos métodos: **Ladrar** y **caminar**.

El primero no recibirá ningún parámetro y el segundo recibirá el número de pasos que queremos andar. Como hemos indicado anteriormente, **self** hace referencia a la instancia de la clase.



Definiendo métodos

```
class Perro:
    # Atributo de clase
    especie = 'mamífero'

    # El método __init__ es llamado al crear el objeto
    def __init__(self, nombre, raza):
        print(f"Creando perro {nombre}, {raza}")

        # Atributos de instancia
        self.nombre = nombre
        self.raza = raza

    def ladra(self):
        print("Guau")

    def camina(self, pasos):
        print(f"Caminando {pasos} pasos")
```

Se puede definir un método con **def y el nombre, y entre () los parámetros de entrada que recibe**, donde siempre tendrá que estar self el primero.



Definiendo métodos

Por lo tanto si creamos un objeto **mi_perro**, podremos hacer uso de sus métodos llamándolos con **.** y el nombre del método. Como si de una función se tratase, pueden recibir y devolver argumentos.



```
mi_perro = Perro("Toby", "Bulldog")
mi_perro.ladra()
mi_perro.camina(10)

# Creando perro Toby, Bulldog
# Guau
# Caminando 10 pasos
```



```
1 class Perro:
2
3     #Atributos de clase
4     especie = "Mamífero"
5
6     #Constructor de la clase
7     def __init__(self, nombre, raza):
8
9         #Atributos de la instancia
10        self.nombre = nombre
11        self.raza = raza
12
13    #Métodos
14    def ladrar(self):
15        print("Este perro ha ladrado :( \nEstá enojado")
16
17
18    def caminar(self, pasos):
19        print(f"Este perro ha caminado {pasos} pasos")
20
21
22 perro1 = Perro("Sammy", "Caniche")
23
24 print(f"Su nombre es: {perro1.nombre}")
25 print(f"Su raza es: {perro1.raza}")
26 print(f"Es un: {perro1.especie}")
27
28 perro1.ladrar()
29 perro1.caminar(5)
30
```

Nuestro código quedaría de la siguiente manera

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
Su raza es: Caniche
Es un: Mamífero
Este perro ha ladrado :(
Está enojado
Este perro ha caminado 5 pasos
```

Métodos especiales

¿Qué son?

Los métodos mágicos de las clases Python son aquellos que comienzan y **terminan con dos caracteres subrayados**. Son de suma importancia ya que muchas de las operaciones que se hacen con clases en Python utilizan estos métodos, como puede ser la creación del objeto (`__init__`) o generar una cadena que los represente (`__str__`).



Métodos especiales

Todos estos métodos tienen un primer parámetro que hace referencia al propio objeto, que por convenio se suele llamar **self**. Siendo el resto de los parámetros del método variables.

Veamos a continuación algunos de estos métodos y cuál es su finalidad. 

Fuente: analyticslane.com

```
1 class Perro:
2
3     #Atributos de clase
4     especie = "Mamifero"
5
6     #Constructor de la clase
7     def __init__(self, nombre, raza):
8
9         #Atributos de la instancia
10        self.nombre = nombre
11        self.raza = raza
12
13    #Métodos
14    def ladrar(self):
15        print("Este perro ha ladrado :( \nEstá enojado")
16
17
18    def caminar(self, pasos):
19        print(f"Este perro ha caminado {pasos} pasos")
20
21
22 perro1 = Perro("Sammy", "Caniche")
23
24 print(f"Su nombre es: {perro1.nombre}")
25 print(f"Su raza es: {perro1.raza}")
26 print(f"Es un: {perro1.especie}")
27
28 perro1.ladrar()
29 perro1.caminar(5)
30
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
Su raza es: Caniche
Es un: Mamifero
Este perro ha ladrado :(
Está enojado
Este perro ha caminado 5 pasos
```



__init__

El método mágico más utilizado en las clases de python es el constructor. Empleado para crear cada una de las instancias de las clases. No tiene una cantidad fija de parámetros, ya que estos dependen de las propiedades que tenga el objeto. Por ejemplo, se puede crear un objeto Vector:

```
class Vector():  
    def __init__(self, data):  
        self._data = data  
  
v = Vector([1,2])  
print(v)
```

str

Posiblemente el segundo método más utilizado, con el que se crea una representación del objeto con significado para las personas.

En el ejemplo anterior, **al imprimir el objeto, se ha visto una cadena que indica el tipo de objeto y una dirección.** Lo que generalmente no tiene sentido para las personas.



__str__

Un problema que se puede
solucionar con el método `__str__`.

```
class Vector():  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values  
are: {self._data}"  
  
v = Vector([1,2])  
print(v)
```



len

Podríamos usar la función len para obtener el número de elementos que tenga el objeto.

Pero si probamos con el objeto nos encontraremos con un error. ☐

```
class Vector():  
  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values are:  
{self._data}"  
  
    def __len__(self):  
        return len(self._data)  
  
v = Vector([1,2])  
len(v)
```



len

Esto es así porque la clase no ha implementado aún el método **len**. Un problema se puede solucionar de tomar fácil al incluir este método.

```
class Vector():  
  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values are:  
{self._data}"  
  
    def __len__(self):  
        return len(self._data)  
  
v = Vector([1,2])  
len(v)
```

getitem



Si se desea que los usuarios puedan leer los elementos mediante el uso de corchetes es necesario implementar el método `__getitem__` en la clase. Este elemento necesita un parámetro adicional, que es la posición del elemento a leer. La función debe retornar el valor leído.

```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values  
are: {self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self,  
pos):
        return self._data[pos]

v = Vector([1,2])
v[1]
```



El método `__getitem__` solo nos sirve para leer, ya que para modificar, se tiene que crear también el método `__setitem__`, el cual veremos a continuación.

setitem



```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values are: {self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self, pos):
        return self._data[pos]

    def __setitem__(self, pos, value):
        self._data[pos] = value

v = Vector([1,2])
v[1] = 20
print(v)
```

Este es el complemento del método anterior. En este caso es necesario pasar dos parámetros adicionales, la posición y el valor a reemplazar.

iter



```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values are: {self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self, pos):
        return self._data[pos]

    def __setitem__(self, pos, value):
        self._data[pos] = value

    def __iter__(self):
        for pos in range(0, len(self._data)):
            yield f"Value[{pos}]: {self._data[pos]}"

v = Vector([1,2])

for vec in v:
    print(vec)
```

Hace que nuestra clase sea iterable, lo que permite usarla por ejemplo en bucles tipo for, es necesario implementar este método. A continuación, podemos ver una implementación de este método en el que devuelve una cadena con el índice y el valor de cada elemento.



iter

```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values are: {self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self, pos):
        return self._data[pos]

    def __setitem__(self, pos, value):
        self._data[pos] = value

    def __iter__(self):
        for pos in range(0, len(self._data)):
            yield f"Value[{pos}]: {self._data[pos]}"

v = Vector([1,2])

for vec in v:
    print(vec)
```

Cuando necesitamos que nuestra clase sea iterable, lo que permite usarla por ejemplo en bucles tipo *for*, es necesario implementar este método. A continuación, podemos ver una implementación en el que devuelve una cadena con el índice y el valor de cada elemento.

CLASES ANIDADAS

Relación entre Clases

Cuando vimos el por qué trabajar por medio de este paradigma, dijimos que era fundamental para **separar a nuestro problema en problemas más pequeños (Clases)**, pero estos “problemas” no van a estar aislados, estarán **relacionados entre sí**. Por ejemplos los **Perros**, suelen ser la mascota de una **Persona**.



Relación entre Clases

Es muy simple. Pero en el próximo slide veremos otra variante de Clases Anidadas.



```
12 def __str__(self):
13     return "Nombre:" + self.nombre + ", Raza:" + self.raza + ", Especie:" + self.especie
14
15
16 #Métodos
17 def ladrar(self):
18     print("Este perro ha ladrado :( \nEstá enojado")
19
20
21 def caminar(self, pasos):
22     print(f"Este perro ha caminado {pasos} pasos")
23
24 class Persona:
25
26     def __init__(self, nombre, apellido, perro):
27         self.nombre = nombre
28         self.apellido = apellido
29         self.perro = perro
30
31
32     def __str__(self):
33         return "Mi nombre es: " + self.nombre + " " + self.apellido + "\n" + self.perro.__str__()
34
35
36 perro1 = Perro("Sammy", "Canicho")
37
38 personal = Persona("Nicolas", "Perez", perro1)
39
40 print(personal)
```

```
perro1 = Perro("Sammy", "Canicho")
personal = Persona("Nicolas", "Perez", perro1)
print(personal)
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
TypeError: can only concatenate str (not "Perro") to str
PS C:\Users\nico\Desktop\Clase xxx & C:\Users\nico\AppData\Local\Microsoft\WindowsApps\python3.9.exe "c:\Users\nico\Desktop\Clase xx\example.py"
Mi nombre es: Nicolas Perez
Nombre:Sammy, Raza:Caniche, Especie:Mamifero
```



Relación entre Clases

```
41 class Sueldo:
42
43     def __init__(self, sueldo):
44         self.sueldo = sueldo
45
46     def __str__(self):
47         return f"\nSUELDO: {self.sueldo}"
48
49
50 class Empleado:
51
52     def __init__(self, nombre, puesto):
53         self.nombre = nombre
54         self.puesto = puesto
55         self.sueldo = Sueldo(1200)
56
57
58     def __str__(self):
59         return f"NOMBRE: {self.nombre}\nPUESTO: {self.puesto}\n" + self.sueldo.__str__()
60
61 s1 = Sueldo(200)
62 emp1 = Sueldo.Empleado("Nico", "Profe")
63
64 print("RESULTADO 1: " + s1.__str__())
65 print("RESULTADO 2: " + emp1.__str__())
```

Esta es otra forma de realizar clases internas a otras clases. No es lo más típico y usado, pero puede ser útil para cuando hay una relación muy estricta entre dos clases.

ENCAPSULAMIENTO

¿De qué se trata?

Es un concepto relacionado con la programación, orientada a objetos, y hace referencia al **ocultamiento de los estados internos de una clase al exterior.**

Dicho de otra manera, encapsular consiste en **hacer que los atributos o métodos internos a una clase no se puedan acceder ni modificar desde fuera,** sino que tan solo el propio objeto pueda acceder a ellos.

***“Python por defecto no oculta los
atributos y métodos de una clase al
exterior 😞”***



Esto es genial para “aprender”, pero en la práctica no nos gustaría que desde la clase Perro le cambien el nombre a una instancia de la clase Sueldo, por dar un ejemplo. Es por eso que es fundamental “defender” a los datos y tornarlos privados.

Visibilidad de los datos

Efectivamente, todo lo que esté privado (**con doble __**) será imposible de acceder desde el exterior de la clase





¿Cómo encapsular en Python?

Hasta el momento sabemos que **la encapsulación es el ocultamiento de datos del estado interno para proteger la integridad del objeto.** En el siguiente ejemplo, CLIENTE es una clase, y hemos encapsulado (al anteponer `__`) la variable `accountNumber`.

```
1 class Customer:
2     def __init__(self):
3         self.__accountNumber = 4321
4     def getAccountNumber(self):
5         return self.__accountNumber
```



¿Cómo encapsular en Python?

Del mismo modo, podemos ocultar el acceso a los métodos. Para eso, necesitamos anteponer el nombre del método con `__` (subrayado doble).

```
1 class Customer:
2     def __init__(self):
3         self.__accountNumber = 4321
4
5     def __processAccount(self):
6         print("Processing Account")
7
8     def getAccountNumber(self):
9         return self.__accountNumber
```